

Relatório de Auditoria de Segurança

Malama

01 de setembro de 2026

branch main · commit e94cccf

Achados	Alta	Média	Baixa	Pontos fortes
9	4	3	2	18

Escopo auditado

- supabase/migrations/ — 155 arquivos de migration (RLS, policies, RPCs, triggers)
- supabase/functions/ — 34 Edge Functions + 5 módulos compartilhados (7.743 linhas)
- src/ — 360 arquivos TypeScript/TSX (rotas, guards, contexts, services, componentes)
- api/share.ts — função serverless da Vercel (página pública de compartilhamento)
- vercel.json, index.html, capacitor.config.ts, codemagic.yaml — cabeçalhos, CSP e CI
- .env.example, .gitignore, scripts/ e histórico do git — gestão de segredos

Nota metodológica

A stack foi detectada antes da auditoria e cada uma das cinco categorias foi traduzida para o equivalente desta arquitetura. Não há ORM nem servidor HTTP próprio: o backend é PostgreSQL com Row Level Security mais Edge Functions em Deno, então “query de listagem sem filtro de tenant” vira “policy ausente ou permissiva” e “handler de rota” vira “Edge Function ou RPC SECURITY DEFINER”. Todo achado abaixo foi verificado no código real, com arquivo e linha; nada é inferido.

Stack detectada e mapeamento das categorias

Camada	O que o projeto usa
Linguagem / build	TypeScript 5.8, React 19, Vite 6, Tailwind 3.
Backend	Supabase (PostgreSQL gerenciado) + 34 Edge Functions em Deno/TypeScript (supabase/functions/*). Uma única função serverless na Vercel (api/share.ts).
ORM / query builder	Nenhum. Acesso direto via @supabase/supabase-js (PostgREST) e RPCs SQL/plpgsql. Sem SQL string-concatenado no cliente.
Autenticação	Supabase Auth (GoTrue), JWT. Papéis (super_admin / rh) em app_metadata; is_super_admin() é o ponto único de verdade.
Isolamento de inquilino	Row Level Security do PostgreSQL em 100% das tabelas, somada a funções SECURITY DEFINER que fazem o gate quando a policy sozinha não expressa a regra.
Frontend	SPA React (react-router 7), embarcada em app nativo via Capacitor 8 (iOS + Android).
Deploy / infra	Vercel (vercel.json com CSP e cabeçalhos), Codemagic (build iOS), pg_cron para jobs. Sem Docker, Helm ou Terraform no repositório.

Como cada categoria foi investigada

1. Banco sem tranca

O mecanismo de isolamento aqui é RLS + funções SECURITY DEFINER. Auditamos as 155 migrations: extraímos toda CREATE TABLE e conferimos ENABLE ROW LEVEL SECURITY; listamos todas as policies com USING(true)/WITH CHECK(true) e verificamos se migrations posteriores as substituíram.

2. Permissão no navegador

Cruzamos cada gate de papel/permissão do frontend (guards.tsx, RhAccessContext, RhLayout, rotas) com a policy ou a RPC correspondente no banco e com a checagem de papel nas Edge Functions.

3. IDOR

Percorremos os 34 handlers de Edge Function um a um e todas as ~110 funções SECURITY DEFINER que recebem um id como parâmetro, procurando ação sobre objeto sem verificação de posse. Policies de storage.objects entram aqui.

4. Chaves expostas

Varredura por padrões de segredo em código, SQL, CI, docs e bundle de produção (dist/), além do histórico do git (git log --diff-filter=A) para arquivos .env, keystores e certificados.

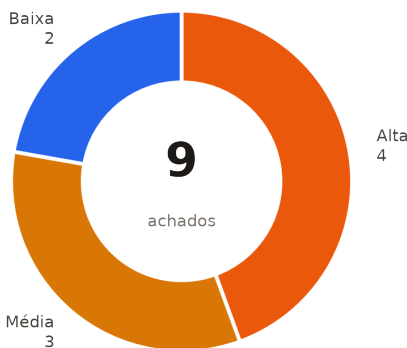
5. Inputs sem tratamento

No frontend: dangerouslySetInnerHTML, innerHTML, eval/new Function e URLs controladas por usuário em href/src. No backend: interpolação de dado do usuário em HTML de e-mail e na página HTML servida por api/share.ts.

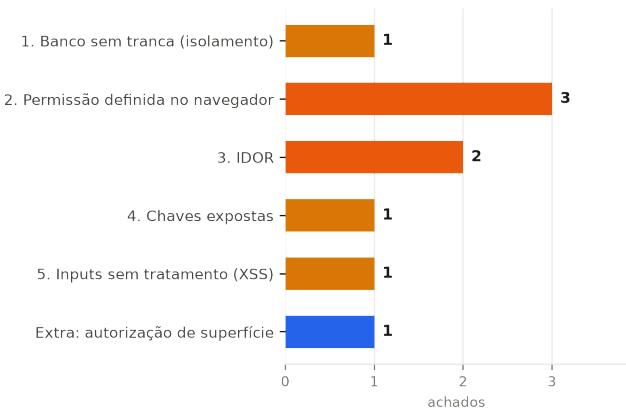
Resumo executivo

Foram identificados **9 achados** — **4 de severidade alta**, **3 média** e **2 baixa**. Nenhum achado crítico: não há tabela sem RLS, não há vazamento entre empresas e nenhum segredo de servidor chega ao bundle do cliente. A base é sólida, e a migration 20260815_security_hardening.sql mostra que o endurecimento já foi feito com método uma vez.

Os achados se concentram em três lugares. O primeiro é o padrão `WITH CHECK (true)`, escrito como se significasse “sem restrição extra” e que na prática libera a linha inteira depois da escrita — é a raiz do IDOR de exames clínicos (F1) e da caixa de notificações aberta (F6). O segundo é a camada de permissões por módulo do portal RH, introduzida em agosto e aplicada ao servidor em apenas quatro superfícies: nas demais, o bloqueio existe só no navegador (F2, F3). O terceiro é o webhook do Strava, único endpoint que escreve no banco com `service_role` sem identificar quem chamou (F4).



Achados por severidade



Achados por categoria (cor = severidade mais grave)

Categoria	Achados	Severidade máxima	IDs
1. Banco sem tranca (isolamento)	1	Média	F6
2. Permissão definida no navegador	3	Alta	F2, F3, F9
3. IDOR	2	Alta	F1, F4
4. Chaves expostas	1	Média	F5
5. Inputs sem tratamento (XSS)	1	Média	F7
Extra: autorização de superfície	1	Baixa	F8

Pontos fortes

Os 18 itens abaixo foram verificados no código e estão corretos. Servem tanto como registro do que está protegido quanto como prova da cobertura da auditoria.

- ✓ **RLS habilitada em 100% das tabelas**
Extração automatizada de todas as CREATE TABLE das 155 migrations: 82 tabelas, 82 com ALTER TABLE ... ENABLE ROW LEVEL SECURITY. Nenhuma tabela nasce destrancada.
- ✓ **Isolamento entre empresas (multi-tenant) sem furos**
Toda policy e toda RPC do portal RH deriva a empresa de `SELECT empresa\_id FROM rh\_usuarios WHERE user\_id = auth.uid()` — nunca de um parâmetro do cliente. Nenhuma consulta de listagem, relatório ou exportação aceita empresa_id vindo do request. Os problemas encontrados (F2, F3) são de módulo dentro da mesma empresa, não de vazamento entre empresas.
- ✓ **Migration 20260815_security_hardening.sql fecha uma classe inteira de falhas**
Derruba todas as policies antigas de doctors e influencers e recria por dono + admin; cria a view public_doctors com security_barrier para o que é realmente público; invalida todos os tokens que haviam sido expostos (linhas 104-107 e 194-198); e reescreve sete RPCs (upsert_reaction, remove_reaction, report_post_fn, follow_user_fn, unfollow_user_fn, log_water_intake, get_patient_name_for_doctor) para rejeitar quando `p\_user\_id IS DISTINCT FROM auth.uid()` — o parâmetro de id deixou de dar poder.
- ✓ **Vínculo médico-paciente exigido antes de qualquer leitura clínica**
generate-patient-ai-report/index.ts:220-231 e activity-health-insights/index.ts:53-64 confirmam consulta existente ou encaminhamento antes de coletar dados do paciente, e respondem 403 quando não há vínculo. sign-prescription/index.ts:34-49 valida que o JWT pertence ao doctor_id informado antes de tocar no certificado.
- ✓ **Checagem de papel no servidor em todas as Edge Functions privilegiadas**
create-rh-user:35-37, create-influencer-user:87-91, invite-lead:75-77, empresa-cobranca:56-58, process-payouts:360-368 e o health-check de webhook-asaas:319-321 exigem app_metadata.role === 'super_admin'. invite-colaborador:73-75 e resend-invite:49-51 exigem RH ativo com a permissão de colaboradores. delete-account:129-134 deriva o usuário do próprio token, então só é possível apagar a si mesmo.
- ✓ **Papel lido de app_metadata, nunca de user_metadata**
is_super_admin() (20260626_rbac_app_metadata.sql:23-28) lê `auth.jwt() -> 'app\_metadata' -> 'role'`, que só o service_role escreve, e é o ponto único chamado por todas as policies administrativas atuais.
- ✓ **URLs de storage validadas antes de virarem href**
doctorPortalService.ts:40-48 (safeLegacyStorageUrl) exige https, origem igual à do projeto Supabase e path começando em /storage/v1/object/. Caminhos internos passam por createSignedUrls (linhas 50-71). Um `javascript:` gravado no banco não chega a href.
- ✓ **Anexos de chat governados por trigger, não só por policy**
protect_chat_message (20260815:415-456) força sender_id e sender_role a partir de auth.uid(), congela todos os campos de conteúdo no UPDATE e valida caminho, MIME e tamanho do anexo. Neutraliza o `WITH CHECK (true)` da policy chat_messages_update_read.
- ✓ **Canal confidencial de assédio desenhado corretamente**
relatos_confidenciais_trabalho não tem nenhuma policy: todo acesso passa por RPC. Leitura e escrita exigem `principal OR 'apuracao' = ANY(permissoes)` (20260836:373-393 e 20260851:206-266), gravam trilha de auditoria e nunca associam user_id ao relato. A porta pública (enviar_relato_publico) tem teto de 20 envios por link por dia e deliberadamente não processa IP.

- ✓ **Proxies de IA com autenticação, gate B2B, cota atômica e allowlist**
caramel-proxy-handler.ts:183-247 exige JWT válido, chama `can_access_mobile_app()` e `consume_edge_quota()` (que usa `pg_advisory_xact_lock, 20260815:388`), limita o corpo a 8 MB, restringe ``action`` e ``model`` a listas fechadas. `rh-agent/index.ts:682-745` segue o mesmo padrão e trata contexto e histórico como dado, nunca como instrução.
- ✓ **Webhook de pagamento com segredo compartilhado e falha fechada**
`webhook-asaas/index.ts:340-347` compara o header `asaas-access-token` com o segredo e registra a tentativa recusada em `integracao_eventos`. Se o segredo não estiver configurado a comparação falha fechada, porque o ramo ``!token`` responde 401 antes.
- ✓ **Credenciais TURN só para participante da consulta e dentro da janela**
`turn-credentials/index.ts:61-82` valida o formato do `room_id`, confirma que o chamador é o paciente ou o médico daquela consulta e exige que o horário esteja dentro da janela de atendimento, antes de emitir credencial efêmera.
- ✓ **Segredos fora do bundle e fora do git**
Só nove variáveis `VITE_` chegam ao cliente, todas públicas por natureza (URL do projeto, anon key, client id do Strava, chave VAPID pública). O bundle em `dist/` não contém `service_role`. O histórico do git nunca recebeu `.env`, `keystore` ou `.pfx`: o único arquivo de ambiente rastreado é o `.env.example`, e `signing/` está no `.gitignore:71`.
- ✓ **platform_settings blindada por nome de chave**
`20260811_segredos_e_integracao.sql:30-46` derruba a policy permissiva anterior e recria a leitura negando chaves cujo nome contenha `key/secret/token/password/senha/api` para quem não é super admin. `20260813` remove a policy redundante que restava.
- ✓ **CSP restritiva nos dois ambientes**
`vercel.json` entrega ``script-src 'self' 'wasm-unsafe-eval'`, object-src 'none', frame-ancestors 'none' e form-action 'self'``. A MESMA política está duplicada como `<meta http-equiv>` em `index.html:13-14`, o que é o detalhe que faz diferença: a WebView do Capacitor não recebe cabeçalho HTTP da Vercel e ficaria descoberta sem isso.
- ✓ **Página pública de compartilhamento escapa o que vem do banco**
`api/share.ts:28-35` define `escapeHtml` e o aplica a `headline`, `subline` e `image_url` antes de montar as meta tags e o corpo (linhas 70-74). O token de indicação passa por `encodeURIComponent` antes de virar href (linha 79).
- ✓ **Frontend sem sink de XSS explorável**
Um único `dangerouslySetInnerHTML` em todo o `src/` (`FoodGuide.tsx:332`), alimentado por `getInsightText()` (linhas 189-196), que devolve string estática do dicionário de `i18n`. Zero ocorrências de `innerHTML`, `outerHTML`, `document.write`, `eval` ou `new Function`.
- ✓ **Trigger protege campos administrativos do profissional**
`protect_doctor_privileged_fields` (`20260815:38-66`) impede que o médico altere o próprio `user_id`, `status`, `platform_fee_percent`, `nivel`, `rating` ou `total_consultations`, e força `status='pending'` no INSERT.

Pontos fracos — os riscos centrais

! A camada de permissão por módulo do RH está pela metade

O modelo introduzido em `20260836` foi aplicado ao servidor em quatro superfícies (`empresa_colaboradores`, `relatos`, `lideranca` e as duas RPCs financeiras) e ficou só no navegador em todo o resto. É o risco mais estrutural do relatório: não é um bug isolado, é uma migração de segurança que parou no meio. F2 e F3.

! WITH CHECK (true) usado como 'sem restrição adicional'

Aparece em `patient_exams_doctor_update` e em `patient_notifications_insert_service`. Nos dois casos a intenção do comentário era mais estreita do que o SQL escrito. Em `chat_messages` o mesmo padrão existe, mas lá um trigger salva a situação — o que mostra que a proteção está sendo aplicada de forma inconsistente. F1 e F6.

! Identificador de objeto vindo do corpo sem autenticação do chamador

strava-webhook decide o que apagar pelo `owner_id` que o requisitante enviou, com cliente `service_role`. É o único endpoint do projeto que escreve no banco sem identificar quem chamou. F4.

! Postura de autenticação das funções depende de flag não versionada

Não existe `supabase/config.toml` no repositório. Se uma função exige JWT ou não é uma decisão tomada na linha de comando do deploy e não registrada em lugar nenhum — o que torna impossível auditar, revisar em PR ou reproduzir o ambiente. Amplifica F4 e F8.

! Sem função de escape reutilizável no lado servidor

`escapeHtml` existe em `api/share.ts` e é bem aplicada lá, mas o módulo de e-mails (`_shared/emails.ts`), que interpola dado de origem pública, não tem equivalente. F7.

! Migrations vulneráveis convivem com as correções no mesmo diretório

A aplicação é manual pelo SQL Editor, sem controle do que já rodou. Quatro arquivos que checam papel por `user_metadata` seguem executáveis. F9.

Achados — visão geral

Severidade	ID	Arquivo:linha	Descrição
Alta	F1	supabase/migrations/ 20260802_acesso_por_tipo_profissional.sql:175-186 supabase/migrations/ 20260422_doctor_panel_v2.sql:357-366 src/services/ doctorPortalService.ts:50-71	Médico libera o bucket inteiro de exames renomeando o próprio perfil 3. IDOR
Alta	F2	src/contexts/ RhAccessContext.tsx:15, 26-28 src/routes/rh/ RhLayout.tsx:238, 259 src/routes/ index.tsx:335-354 +4 arquivo(s)	Permissões por módulo do portal RH existem só no navegador 2. Permissão definida no navegador
Alta	F3	supabase/migrations/ 20260842_relatorios_valor_probatorio.sql:242-275	rh_certificado_colaboradores() contorna a policy do módulo 'colaboradores' 2. Permissão definida no navegador
Alta	F4	supabase/functions/strava-webhook/ index.ts:24-38, 40-57, 61-71	strava-webhook aceita POST sem autenticação e age sobre o owner_id do corpo 3. IDOR
Média	F5	.env.example:43, 47, 50	Segredo do webhook Strava versionado em .env.example 4. Chaves expostas
Média	F6	supabase/migrations/ 20260423_patient_notifications.sql:43-46	Qualquer usuário autenticado insere notificações na caixa de qualquer paciente 1. Banco sem tranca (isolamento)
Média	F7	supabase/functions/_shared/ emails.ts:42-47, 54, 102, 145-156 supabase/functions/self-register-empresa/ index.ts:49, 87 supabase/functions/invite-colaborador/ index.ts:195-196 +1 arquivo(s)	E-mails transacionais montam HTML sem escapar dado de origem externa 5. Inputs sem tratamento (XSS)
Baixa	F8	supabase/functions/send-consultation-reminders/ index.ts:247 supabase/functions/send-glp1-notifications/ index.ts:99 supabase/functions/send-rh-reminders/ index.ts:210-212	Funções de cron não verificam quem as chamou Extra: autorização de superfície
Baixa	F9	supabase/migrations/ 20260601_empresas_b2b.sql:21-24 supabase/migrations/ 20260504_admin_profiles_policy.sql:6, 30 supabase/migrations/ 20260621_admin_update_setting_rpc.sql:11 +3 arquivo(s)	user_metadata decidia privilégio — em migrations antigas E em 3 policies vivas sem arquivo 2. Permissão definida no navegador

Achados detalhados

Agrupados pela categoria da auditoria. Cada achado traz o trecho exato do código, por que é explorável, o impacto, as condições de explorabilidade e a correção sugerida.

1. Banco sem tranca (isolamento)

Média

F6 — Qualquer usuário autenticado insere notificações na caixa de qualquer paciente

Localização

supabase/migrations/20260423_patient_notifications.sql:43-46

Evidência

supabase/migrations/20260423_patient_notifications.sql:42-46

```
-- Service role pode inserir (usado pelos triggers SECURITY DEFINER)
CREATE POLICY "patient_notifications_insert_service"
ON public.patient_notifications
FOR INSERT
WITH CHECK (true);
```

Por que é explorável

A intenção declarada no comentário é 'service role pode inserir'. Mas a policy não tem cláusula TO, então vale para todos os papéis, incluindo authenticated e anon — e o Supabase concede INSERT nessas tabelas por padrão (não há REVOKE para patient_notifications em nenhuma migration). O WITH CHECK (true) não amarra o user_id ao auth.uid().

Vale notar o contraste: a tabela irmã doctor_notifications faz certo — 20260422_doctor_panel_notifications.sql:60 usa WITH CHECK (false), permitindo insert apenas via SECURITY DEFINER. O comentário nessa linha é literalmente 'apenas service_role insere'. A regra correta já existe no projeto; só não foi aplicada aqui.

Além disso, a policy era desnecessária desde o início: os triggers que a justificam (notify_patient_on_chat_opened, linha 50) são SECURITY DEFINER e escrevem como o dono da função, não como o usuário.

Impacto

Phishing dentro do produto. O atacante escolhe title, body e o JSONB `data` de uma notificação exibida ao paciente com a credibilidade da própria plataforma — por exemplo, tipo 'chat_opened' apontando para um chat que não existe, ou um aviso clínico falso. Também permite poluir e inundar a caixa de notificações de qualquer usuário.

Condições de explorabilidade

Uma chamada supabase.from('patient_notifications').insert({...}) com qualquer sessão autenticada do app. Sem pré-condição. A leitura permanece corretamente restrita ao dono (policy de SELECT na linha 31), então não há vazamento — o problema é de escrita.

Correção sugerida

Trocar por `WITH CHECK (false)`, espelhando doctor_notifications, ou por `TO service\_role WITH CHECK (true)`. Confirmar que os triggers SECURITY DEFINER continuam inserindo (eles não são afetados por RLS quando o dono é superusuário/postgres) e adicionar REVOKE INSERT ON public.patient_notifications FROM

anon, authenticated.

2. Permissão definida no navegador

Alta

F2 — Permissões por módulo do portal RH existem só no navegador

Localização

src/contexts/RhAccessContext.tsx:15, 26-28

src/routes/rh/RhLayout.tsx:238, 259

src/routes/index.tsx:335-354

supabase/migrations/20260731_absenteismo_ambulatorio.sql:66-69

supabase/migrations/20260803_link_por_setor.sql:503-517

supabase/migrations/20260847_plano_expoe_origem_lideranca.sql:51

supabase/migrations/20260619_empresa_billing.sql:62-66

Evidência

src/contexts/RhAccessContext.tsx:15, 26-28

```
can: permissao => acesso.principal || acesso.permissoes.includes(permissao),
...
export const RhPermissionGate: React.FC<{...}> = ({ permissao, children }) => {
  const { can } = useRhAccess();
  return can(permissao) ? <{children}> : <RhHomeRedirect />;
};
```

supabase/migrations/20260731_absenteismo_ambulatorio.sql:66-69

```
CREATE POLICY "rh reads own afastamentos"
ON empresa_afastamentos FOR SELECT TO authenticated
USING (empresa_id IN (SELECT empresa_id FROM rh_usuarios WHERE user_id = auth.uid()));
```

supabase/migrations/20260803_link_por_setor.sql:512-517

```
SELECT e.id, e.nome, e.cnpj INTO v_empresa
FROM empresas e
WHERE e.id IN (SELECT empresa_id FROM rh_usuarios WHERE user_id = auth.uid())
LIMIT 1;

IF v_empresa.id IS NULL THEN RETURN NULL; END IF;
```

Por que é explorável

A migration 20260836 introduziu o modelo de permissões por módulo (colaboradores, saude_mental, absenteismo, plano_acao, importar, financeiro, compliance, empresa, usuarios, apuracao) e o helper rh_tem_permissao(). Esse helper foi de fato aplicado nas policies de empresa_colaboradores (20260836:196-221), nas RPCs de relatos confidenciais, nas de liderança e nas duas financeiras de 20260843.

Fora dessas, o servidor continua perguntando apenas 'este usuário é um rh_usuarios desta empresa?'. Ficaram sem o gate de módulo:

- empresa_afastamentos (SELECT/INSERT/DELETE) — CID e dias de afastamento;
- empresa_ambulatorio, empresa_planos_acao, empresa_compliance_docs, empresa_certificados_esg, empresa_ingestao_lotes, empresa_relatorios_emitidos, psychosocial_campaigns, empresa_faturas — todas com policy só de empresa;
- RPCs rh_relatorio_psicossocial, rh_matriz_psicossocial, rh_relatorio_jss,

rh_absenteismo_resumo, rh_ambulatorio_resumo, rh_listar_planos_acao,
rh_criar/atualizar/excluir_plano_acao, rh_lancar_afastamento,
rh_lancar_ambulatorio, rh_vincular_cpfs, rh_ingerir_afastamentos.

O bloqueio é só a UI: RhLayout filtra as abas e RhPermissionGate redireciona a rota. Quem tem sessão do portal já tem a anon key no bundle e pode chamar supabase.from('empresa_afastamentos').select('*') ou supabase.rpc('rh_relatorio_psicossocial', ...) direto do console do navegador.

Impacto

Escalada horizontal dentro da empresa. Um membro com papel 'financeiro' lê o relatório psicossocial (WHO-5/JSS por setor), a lista de afastamentos com CID e os atendimentos de ambulatorio — exatamente o dado que o desenho do produto separa por módulo. O modelo de permissão da tela de Usuários passa a ser uma promessa que o servidor não cumpre.

Condições de explorabilidade

Qualquer conta de equipe do RH criada via invite-rh-user com permissões restritas. Nenhuma condição especial: basta a sessão legítima e uma chamada direta ao PostgREST/RPC. O corte k-anônimo ($k \geq 5$) segue valendo e limita a reidentificação individual, mas não impede a leitura do módulo inteiro.

Correção sugerida

Adicionar `public.rh\_tem\_permissao('<modulo>')` a cada policy e a cada RPC listada acima, no mesmo formato já usado em empresa_colaboradores. Aproveitar para incluir `AND ativo` no subselect de rh_usuarios — várias policies e RPCs não filtram por conta ativa (a desativação hoje depende de rh_atualizar_usuario zerar o user_id, que é uma proteção indireta).

Alta

F3 — rh_certificado_colaboradores() contorna a policy do módulo 'colaboradores'

Localização

supabase/migrations/20260842_relatorios_valor_probatorio.sql:242-275

Evidência

supabase/migrations/20260842_relatorios_valor_probatorio.sql:242-275

```
CREATE OR REPLACE FUNCTION rh_certificado_colaboradores()
RETURNS TABLE (
  colaborador_id UUID, nome TEXT, setor TEXT, funcao TEXT,
  data_adicao TIMESTAMPTZ, data_ativacao TIMESTAMPTZ,
  data_saida TIMESTAMPTZ, status TEXT
) AS $$
WITH emp AS (
  SELECT empresa_id FROM rh_usuarios WHERE user_id = auth.uid() LIMIT 1
),
...
FROM empresa_colaboradores ec
LEFT JOIN public.profiles p ON p.id = ec.user_id
LEFT JOIN auth.users u ON u.id = ec.user_id
WHERE ec.empresa_id = (SELECT empresa_id FROM emp)
$$ LANGUAGE sql SECURITY DEFINER SET search_path = public STABLE;
```

Por que é explorável

Esta é a única tabela onde a permissão de módulo FOI aplicada em RLS (20260836:196-201 exige `rh_tem_permissao('colaboradores')`). Mas esta RPC é SECURITY DEFINER e ignora RLS por definição, e o único guard que ela faz é 'existe uma linha em `rh_usuarios` com meu `user_id`'. Ela nem sequer checa ``ativo``. O resultado é a lista nominal completa de colaboradores da empresa — nome, setor, função, datas de entrada e saída — para quem a policy da tabela acabou de negar.

O mesmo padrão aparece em `rh_listar_planos_acao` (20260847_plano_expoe_origem_lideranca.sql:51), que também é SECURITY DEFINER sem checagem de módulo nem de conta ativa.

Impacto

Anula, na prática, a única policy de módulo implementada do lado servidor. Exposição da nominata de colaboradores (dado pessoal identificável) a membros do RH que a empresa decidiu não dar esse acesso.

Condições de explorabilidade

Uma chamada `supabase.rpc('rh_certificado_colaboradores')` com a sessão de qualquer `rh_usuarios` da empresa, inclusive uma conta desativada cujo `user_id` não tenha sido zerado.

Correção sugerida

Adicionar no topo da função o mesmo guard das RPCs de relato: selecionar de `rh_usuarios` com ``AND ativo AND (principal OR 'colaboradores' = ANY(permissoes))`` e levantar exceção quando não houver linha. Repetir em `rh_listar_planos_acao` com o módulo `'plano_acao'`.

[Baixa](#)

F9 — user_metadata decidia privilégio — em migrations antigas E em 3 policies vivas sem arquivo

Localização

`supabase/migrations/20260601_empresas_b2b.sql:21-24`

`supabase/migrations/20260504_admin_profiles_policy.sql:6,30`

`supabase/migrations/20260621_admin_update_setting_rpc.sql:11`

`supabase/migrations/20260621_payouts_doctor_policy.sql:29-30`

`supabase/migrations/20260621_platform_settings_admin_policy.sql:35,38`

(sem arquivo — 3 policies vivas, achadas só em produção)

Evidência

`supabase/migrations/20260621_payouts_doctor_policy.sql:27-30`

```
CREATE POLICY "Admins manage payouts"
ON payouts FOR ALL
USING ((auth.jwt() -> 'user_metadata' ->> 'role') = 'super_admin')
WITH CHECK ((auth.jwt() -> 'user_metadata' ->> 'role') = 'super_admin');
```

`supabase/migrations/20260601_empresas_b2b.sql:21-24`

```
-- Helper: o JWT do chamador é de um super_admin?
CREATE OR REPLACE FUNCTION is_super_admin()
RETURNS BOOLEAN AS $$
  SELECT (auth.jwt() -> 'user_metadata' ->> 'role') = 'super_admin';
$$ LANGUAGE sql STABLE;
```

`supabase/migrations/20260626_rbac_app_metadata.sql:5-8 (a correção)`

```
-- PROBLEMA: o papel (super_admin/rh) era lido de user_metadata, que o próprio
-- usuário altera via supabase.auth.updateUser({ data: { role } }) no navegador.
-- Isso permitia auto-promoção a super_admin (acesso a todos os dados).
-- app_metadata (raw_app_meta_data) só é gravável por service_role / admin API.
```

pg_policies em produção — sem arquivo correspondente em nenhum lugar do repo

```
CREATE POLICY "Admin can view all consultations" ON consultations FOR SELECT
  USING (((auth.jwt() -> 'user_metadata') ->> 'role') = 'super_admin');
CREATE POLICY "Admin can update all consultations" ON consultations FOR UPDATE
  USING (((auth.jwt() -> 'user_metadata') ->> 'role') = 'super_admin');
CREATE POLICY plan_prices_admin_write ON plan_prices FOR ALL
  USING (((auth.jwt() -> 'user_metadata') ->> 'role') = 'super_admin');
```

Por que é explorável

user_metadata é gravável pelo próprio usuário (supabase.auth.updateUser). Qualquer verificação de privilégio que o leia é auto-promoção. A migration 20260626_rbac_app_metadata.sql fechou isso corretamente: redefiniu is_super_admin() para ler app_metadata (linha 23-28) e refez, com o mesmo nome e a mesma assinatura, todas as policies e RPCs afetadas — incluindo 'Admins manage payouts' (linha 96-99), 'Admins can manage settings' (90-93), 'Admins can view all profiles' (31-34), admin_get_all_users() (37-66) e admin_update_setting() (70-87).

O achado não é uma brecha aberta hoje: é o risco de reintrodução. Os CINCO arquivos vulneráveis continuam no diretório de migrations, e este projeto aplica migrations manualmente pelo SQL Editor, arquivo a arquivo, sem tabela de controle de versão aplicada. Executar qualquer um deles depois de 20260626 — num ambiente novo, num restore, ou por engano de ordenação — recria a policy vulnerável em silêncio, sem erro.

CONFIRMADO NA PRÁTICA em 01/09/2026: ao aplicar a correção deste relatório, o teste de smoke do bloco 4 (que varre pg_policies e pg_proc inteiros, não só os cinco arquivos catalogados) achou TRÊS policies vivas com o mesmo predicado — 'Admin can view all consultations', 'Admin can update all consultations' e 'plan_prices_admin_write' — que não existem em NENHUM arquivo do repositório, nem em supabase/migrations/, nem nos supabase-*.sql anteriores à pasta de migrations. Foram aplicadas diretamente no banco em algum momento não documentado. Ou seja: a classificação original — 'risco de reintrodução, não brecha ativa' — estava certa para os cinco arquivos catalogados, mas incompleta: para consultations e plan_prices, a auto-promoção via user_metadata era EXPLORÁVEL AGORA, não uma possibilidade futura. As três foram corrigidas na mesma migração (bloco 4) e o smoke test hoje passa limpo.

Impacto

O pior dos cinco é 20260601_empresas_b2b.sql:21-24: ali não é uma policy, é a DEFINIÇÃO de is_super_admin() — o ponto único que todas as policies administrativas consultam. Reaplicar aquele arquivo reabre empresas, payouts, platform_settings e profiles de uma vez só, sem erro nenhum. Se reintroduzido: escalada a super_admin por qualquer usuário do app, com controle sobre platform_settings (valores de repasse por nível, taxa de transação) e sobre a tabela payouts. Para os cinco arquivos catalogados, o risco era de processo, não de estado corrente — mas ver a nota acima: para consultations (dados de consulta/telemedicina) e plan_prices (preços dos planos), o mesmo problema era estado corrente, não hipótese, até ser corrigido nesta rodada.

Condições de explorabilidade

Para os cinco arquivos catalogados: não explorável no estado em que a auditoria original encontrou o banco, desde que 20260626 tenha sido a última a rodar sobre esses objetos — torna-se explorável em qualquer reaplicação fora de ordem. Para as três policies descobertas depois (consultations x2, plan_prices x1): eram exploráveis sem nenhuma condição — bastava supabase.auth.updateUser({ data: { role: 'super_admin' } }) no navegador. Corrigido nesta migração.

Correção sugerida

Reescrever os quatro arquivos legados para chamar `is_super_admin()` (tornando-os idempotentes e seguros em qualquer ordem) ou marcá-los como superados com um cabeçalho e um ``RAISE EXCEPTION`` de guarda. Adotar uma tabela de migrations aplicadas para que a ordem deixe de ser responsabilidade humana. Adicionar um teste de smoke que verifique que nenhuma policy em `pg_policies` referencia `user_metadata`.

3. IDOR

Alta

F1 — Médico libera o bucket inteiro de exames renomeando o próprio perfil

Localização

supabase/migrations/20260802_acesso_por_tipo_profissional.sql:175-186

supabase/migrations/20260422_doctor_panel_v2.sql:357-366

src/services/doctorPortalService.ts:50-71

Evidência

supabase/migrations/20260802_acesso_por_tipo_profissional.sql:175-186

```
CREATE POLICY "doctor_exams_read" ON storage.objects
FOR SELECT TO authenticated
USING (
  bucket_id = 'patient-exams'
  AND EXISTS (
    SELECT 1 FROM public.patient_exams pe
    JOIN public.doctors d ON d.id = pe.doctor_id
    WHERE d.user_id = auth.uid()
    AND COALESCE(d.tipo_profissional, 'medico') = 'medico'
    AND pe.file_url LIKE '%' || name -- <= `name` NU
  )
);
```

supabase/migrations/20260422_doctor_panel_v2.sql:357-366

```
CREATE POLICY "patient_exams_doctor_update" ON public.patient_exams
FOR UPDATE TO authenticated
USING (
  EXISTS (SELECT 1 FROM public.doctors d
    WHERE d.id = patient_exams.doctor_id AND d.user_id = auth.uid())
)
WITH CHECK (true);
```

Por que é explorável

``name`` está sem qualificação, e a tabela ``doctors`` — que está no FROM da própria subquery — TEM uma coluna ``name``. Em SQL, um nome de coluna não qualificado se liga primeiro ao FROM do nível mais interno. Logo, a policy não compara com ``storage.objects.name``: compara com o NOME DO MÉDICO.

Verificado em PostgreSQL 16, não deduzido. Com o mesmo dado, a versão com ``name`` nu devolve 0 objetos e a versão com ``objects.name`` qualificado devolve 1.

Daí saem duas consequências, e a segunda é a grave:

1. Com um nome normal ("Dra Ana Souza"), a condição nunca casa. O médico não abre exame NENHUM — nem dos próprios pacientes. A visualização de exames do portal está quebrada hoje, e em silêncio: `signStoragePaths` engole o erro de `createSignedUrls` e devolve `file_url = null`, então a tela mostra um link inerte em vez de um erro.

2. O EXISTS deixou de ser correlacionado ao objeto. A condição virou "existe algum exame meu cujo file_url termina com o meu próprio nome" — uma pergunta que não fala do arquivo sendo pedido. Se ela for verdadeira, é verdadeira para TODOS os objetos do bucket. E o médico controla os dois lados: basta pôr em `doctors.name` um sufixo de qualquer arquivo dele — "pdf" serve. A policy doctors_own_update (20260815:31-33) permite editar o próprio nome, e protect_doctor_privileged_fields (20260815:38-66) protege user_id, status, nível, rating e platform_fee_percent — mas não o nome.

Confirmado no mesmo teste: com o nome "Dra Ana Souza", 0 de 3 objetos visíveis; após UPDATE do nome para "pdf", 3 de 3.

Separadamente, patient_exams_doctor_update termina em WITH CHECK (true): o USING filtra a linha ANTES da escrita, o WITH CHECK valida DEPOIS. O médico reatribui uma linha sua para outro paciente ou outro profissional. Não é o vetor de leitura, mas passa a ser se alguém corrigir a policy de storage só qualificando a coluna, sem ancorar na pasta.

Impacto

Leitura de todos os exames clínicos (PHI) da plataforma por qualquer médico aprovado, ao custo de um UPDATE no próprio perfil. Não é preciso conhecer UUID de vítima nem nome de arquivo: a policy libera o bucket e o storage lista. Violação de LGPD art. 11 e do sigilo profissional. Em paralelo, uma funcionalidade do portal está inoperante desde 20260802.

Condições de explorabilidade

Conta de profissional com status approved, tipo_profissional medico e ao menos um exame vinculado — o fluxo normal do portal já produz isso. Depois, um único UPDATE em doctors.name. Sem feature flag e sem config insegura necessária.

Correção sugerida

1) Qualificar a coluna (storage.objects.name) e trocar o LIKE de sufixo por igualdade. 2) Ancorar o objeto na pasta do paciente da linha: (storage.foldername(storage.objects.name))[1] = pe.patient_id::text — o upload sempre grava em <patient_id>/... e a policy de INSERT obriga a pasta a ser o auth.uid(), então é um vínculo que o médico não move. 3) Preservar a restrição tipo_profissional = 'medico' de 20260802, senão a correção alarga o acesso a psicólogos. 4) WITH CHECK igual ao USING em patient_exams_doctor_update, mais trigger congelando patient_id/doctor_id/file_url. 5) Varrer as demais policies de storage.objects pelo mesmo padrão de sombreamento de coluna.

Alta

F4 — strava-webhook aceita POST sem autenticação e age sobre o owner_id do corpo

Localização

supabase/functions/strava-webhook/index.ts:24-38, 40-57, 61-71

Evidência

supabase/functions/strava-webhook/index.ts:24-35

```
// -- POST: Evento de atividade ou deauthorização -----
if (req.method === 'POST') {
  // Strava exige resposta em 2 segundos – processamento em background
  const payload = await req.json();

  // Responder imediatamente antes de processar
  const responsePromise = new Response('EVENT_RECEIVED', { status: 200 });

  // Processar em background (não bloqueia a resposta)
  processEvent(payload).catch(...);

  return responsePromise;
}
```

supabase/functions/strava-webhook/index.ts:46-57

```
const ownerId    = payload.owner_id as number;

// -- Deauthorização: remover conexão -----
if (objectType === 'athlete') {
  const updates = payload.updates as Record<string, string> | undefined;
  if (updates?.authorized === 'false') {
    await supabase
      .from('strava_connections')
      .delete()
      .eq('strava_athlete_id', ownerId);
  }
  return;
}
```

Por que é explorável

O ramo GET valida STRAVA_VERIFY_TOKEN (linha 16) — é o handshake de assinatura. O ramo POST, que é o que efetivamente escreve no banco, não valida absolutamente nada: nem token, nem assinatura, nem origem. O cliente Supabase é instanciado com SUPABASE_SERVICE_ROLE_KEY (linha 41), então processEvent roda com poder total, acima de qualquer RLS.

Todo o roteamento usa campos do corpo enviado pelo atacante: `owner\_id` decide de qual usuário a conexão será apagada, e `object\_id` decide qual atividade será buscada e gravada. É o padrão clássico de IDOR — identificador de objeto vindo do request, sem verificação de quem está chamando.

Impacto

Negação de serviço direcionada: POST com {object_type:'athlete', owner_id:N, updates:{authorized:'false'}} apaga a conexão Strava do usuário N; iterando N apaga todas. Além disso, o mesmo endpoint permite forçar a re-sincronização e a gravação de atividades em activities e flow_stats de terceiros — dados que alimentam os relatórios clínicos de IA (generate-patient-ai-report e activity-health-insights) e, portanto, contaminam conduta médica.

Condições de explorabilidade

Depende da flag verify_jwt com que a função foi publicada — e o repositório NÃO declara essa flag (não existe supabase/config.toml). Nos dois cenários há exposição: com --no-verify-jwt (obrigatório para o Strava alcançar a função, e é o que a própria webhook-asaas documenta na linha 310) qualquer pessoa na internet explora; com verify_jwt padrão, qualquer usuário logado do app explora. O apagamento de conexões (object_type 'athlete') não requer nem token do Strava.

Correção sugerida

RESOLVIDO POR DESCOMISSIONAMENTO, não por autenticação (decisão de produto de 01/09/2026): a plataforma passou a cobrir atividade física por Apple HealthKit e Google Health Connect, então proteger um endpoint que ninguém mais deveria chamar deixou de fazer sentido — desligar é mais seguro que autenticar

algo obsoleto. strava-webhook agora devolve 410 Gone incondicionalmente, sem ler o corpo da requisição nem tocar em strava_connections/activities. strava-oauth e strava-oauth-callback (que abriam novas conexões) foram desativadas junto, pelo mesmo motivo. strava-sync e strava-refresh-token permanecem ativas apenas para quem já tinha o Strava conectado, até a remoção completa da integração (tarefa separada: 18 arquivos de frontend, as 5 Edge Functions, migrations e a decisão sobre o destino de strava_connections e das activities já sincronizadas).

4. Chaves expostas

Média

F5 — Segredo do webhook Strava versionado em .env.example

Localização

.env.example:43,47,50

Evidência

.env.example:43-50

```
# Configure via: supabase secrets set STRAVA_CLIENT_ID=227202 STRAVA_CLIENT_SECRET=xxx STRAVA_VERIFY_TOKEN=nura_strava_2026
#
# STRAVA_CLIENT_ID      = 227202
# STRAVA_CLIENT_SECRET = <seu client secret do portal Strava – não committar>
# STRAVA_VERIFY_TOKEN   = nura_strava_2026
STRAVA_CLIENT_ID=227202
STRAVA_CLIENT_SECRET=
STRAVA_VERIFY_TOKEN=nura_strava_2026
```

Por que é explorável

STRAVA_CLIENT_SECRET foi corretamente deixado em branco, com aviso explícito. STRAVA_VERIFY_TOKEN, não: o valor real de produção aparece três vezes no arquivo, inclusive como valor efetivo na linha 50 e dentro do comando de provisionamento da linha 43. O arquivo é rastreado pelo git desde 37f6380 (14/03/2026) — confirmado com `git log --diff-filter=A -- '\*.env\*'`.

Esse token é o único fator que autentica o handshake de assinatura do webhook (strava-webhook/index.ts:16). Ele não é um placeholder: segue o padrão de nome do projeto ('nura' era o nome anterior do produto) e casa com o comando de provisionamento documentado.

Impacto

Quem tem acesso ao repositório (ou a qualquer fork/clone/backup dele) pode registrar sua própria subscription no endpoint e responder ao handshake de validação. Combinado com F4, remove a última barreira de obscuridade do webhook. Vale também como sinal de processo: não existe validação de startup que recuse um segredo igual ao default publicado.

Condições de explorabilidade

Imediata para qualquer pessoa com leitura do repositório. Não exige nenhuma condição de configuração.

Correção sugerida

Ficou parcialmente superado pela decisão de descomissionar o webhook (ver F4): o segredo que vazou não protege mais nada, porque strava-webhook não lê STRAVA_VERIFY_TOKEN em nenhuma circunstância — a função devolve 410 antes de examinar qualquer header ou query string. O valor não precisa mais ser rotacionado por urgência de segurança. .env.example foi limpo (não cita mais STRAVA_VERIFY_TOKEN nem STRAVA_WEBHOOK_SECRET). Continua valendo, como prevenção geral: guard de inicialização nas Edge

Functions que recusa segredo igual a um valor conhecido de exemplo, e scanner de segredo no CI (P3).

5. Inputs sem tratamento (XSS)

Média

F7 — E-mails transacionais montam HTML sem escapar dado de origem externa

Localização

supabase/functions/_shared/emails.ts:42-47,54,102,145-156

supabase/functions/self-register-empresa/index.ts:49,87

supabase/functions/invite-colaborador/index.ts:195-196

supabase/functions/webhook-asaas/index.ts:290-299

Evidência

supabase/functions/_shared/emails.ts:42-47

```
const paragraphs = bodyParagraphs
  .map(
    (p) =>
      `
```

supabase/functions/_shared/emails.ts:145-150

```
export function activationEmailHtml(empresaNome: string, ctaUrl: string): string {
  return brandedEmailHtml({
    preheader: `${empresaNome} liberou o benefício Malama para você.`,
    heading: `Seu benefício Malama foi <em ...>liberado</em>`,
    bodyParagraphs: [
      `A <strong>${empresaNome}</strong> adicionou você ao benefício de saúde Malama.`,
    ],
  });
}
```

supabase/functions/self-register-empresa/index.ts:49,87

```
const nomeEmpresa = String(body.empresa ?? '').trim();
...
  nome: nomeEmpresa, // gravado sem sanitização de HTML
```

Por que é explorável

brandedEmailHtml interpola heading, bodyParagraphs, eyebrow, footnote, ctaText e ctaUrl direto no template, sem escape. Não existe nenhuma função de sanitização no módulo — o projeto tem uma (escapeHtml em api/share.ts:28-35), mas ela vive do outro lado e não é reutilizada aqui.

O valor que chega até esse HTML nem sempre é confiável. self-register-empresa é uma função PÚBLICA e não autenticada: ela grava empresas.nome com o que veio do formulário, sem qualquer filtro de markup (a única validação é 'não vazio'). Esse mesmo campo depois vai para activationEmailHtml em invite-colaborador:196 e resend-invite:79, e para o alerta ao administrador em webhook-asaas:295.

É importante ser preciso sobre o alcance: cliente de e-mail moderno não executa JavaScript, então isto não é XSS clássico. O que se obtém é injeção de HTML — âncoras, imagens de rastreo e blocos de texto forjados dentro de uma mensagem com a marca Malama.

Impacto

Phishing de alta credibilidade: um `<a href="https://site-do-atacante">Ativar meu benefício</a>` embutido no corpo de um e-mail legítimo da Malama, enviado da infraestrutura da Malama, para o endereço corporativo do colaborador. O alerta de inadimplência em webhook-asaas leva o mesmo texto para a caixa do administrador da plataforma.

Condições de explorabilidade

Condicional. Empresa vinda do autocadastro nasce em status `'em_configuracao'` e, nesse estado, invite-colaborador só grava rascunho e não dispara e-mail (invite-colaborador/index.ts:128-141). É preciso que o super admin ative a empresa comercialmente para o envio ocorrer — o que é, justamente, o fluxo esperado de um lead que fez autocadastro. O caminho do webhook-asaas exige, além disso, uma fatura vencida.

Correção sugerida

Criar um `escapeHtml` no `_shared` (ou mover o de `api/share.ts` para lá) e aplicá-lo a todo valor dinâmico antes da interpolação. Onde o markup é intencional (o `<em>` do heading, o `<strong>` dos parágrafos fixos), montar a string já escapando só a parte variável. Validar `ctaUrl` contra uma allowlist de origem antes de colocá-lo em href. No cadastro, rejeitar `'<'` e `'>'` em nome de empresa.

Extra: autorização de superfície

Baixa

F8 — Funções de cron não verificam quem as chamou

Localização

`supabase/functions/send-consultation-reminders/index.ts:247``supabase/functions/send-glp1-notifications/index.ts:99``supabase/functions/send-rh-reminders/index.ts:210-212`

Evidência

`supabase/functions/send-consultation-reminders/index.ts:247`

```
Deno.serve(async (_req) => {
  try {
    const nowMs = Date.now();
```

`supabase/functions/send-rh-reminders/index.ts:210-212`

```
Deno.serve(async (req) => {
  // `?forcar=1` ignora o dia da semana – para testar sem esperar a segunda.
  const url = new URL(req.url);
  const forcar = url.searchParams.get('forcar') === '1';
```

Por que é explorável

Três das funções acionadas por `pg_cron` descartam a requisição inteira (o parâmetro se chama `_req`) ou leem só a query string, sem conferir o Authorization. Isso destoa do padrão correto adotado no resto do projeto: `expire-credits:19-22`, `compute-clinical-outcomes:33-38`, `generate-empirical-cases:422-439` e `process-payouts:360-368` todas comparam o token com o `SERVICE_KEY` ou exigem `super_admin`.

`send-rh-reminders` é a mais sensível das três porque expõe `?forcar=1`, que ignora a checagem de dia da semana.

Impacto

Disparo forçado de rotinas de envio: lembretes de consulta, notificações de dose GLP-1 e o digest semanal do RH. O dedupe interno (`consultation_reminders_sent`, `notification_sent`, `rh_lembretes_enviados`) limita o volume a um envio por evento, então o efeito é antecipação e ruído, não flood. `send-consultation-reminders` também executa `processNoShows`, que marca consultas como `no_show` e devolve crédito — rodá-la fora de hora antecipa essa transição de estado.

Condições de explorabilidade

Depende de `verify_jwt`, que o repositório não versiona (não há `supabase/config.toml`). Com o padrão `verify_jwt=true`, qualquer usuário logado consegue invocar. Com `--no-verify-jwt`, qualquer pessoa. O impacto contido pelo dedupe é o que mantém isto em severidade baixa.

Correção sugerida

Aplicar o guard de `expire-credits/index.ts:19-22` nas três funções e versionar `supabase/config.toml` declarando `verify_jwt` por função, para que a postura pare de depender de uma flag de linha de comando não registrada.

Recomendações priorizadas

P1 é o que deve ser corrigido antes de qualquer outra coisa: exposição de dado clínico de terceiros e escrita não autenticada. P2 restaura garantias que o produto já promete na interface. P3 é endurecimento e prevenção de regressão.

Prio	Ação	Detalhe
P1	Corrigir o IDOR de exames clínicos	WITH CHECK equivalente ao USING em patient_exams_doctor_update, trigger congelando patient_id/doctor_id/file_url, e reescrita de doctor_exams_read para amarrar a pasta do objeto ao paciente. É o único achado que expõe PHI de terceiros. (F1)
Feito	strava-webhook descomissionado, não autenticado	Decisão de produto (01/09/2026): a plataforma cobre atividade física por Apple HealthKit e Google Health Connect, então o webhook foi desligado (410 incondicional) em vez de ganhar autenticação. strava-oauth e strava-oauth-callback desligados junto. (F4)
Feito	STRAVA_VERIFY_TOKEN — urgência de rotação superada	O segredo versionado desde março de 2026 não é mais lido por nenhum código: strava-webhook devolve 410 antes de examinar qualquer header. .env.example já foi limpo. Rotação no portal Strava segue recomendada, mas não é mais P1. (F5)
P2	Levar as permissões por módulo do RH para o servidor	Aplicar rh_tem_permissao() em todas as policies empresa_* e em todas as RPCs rh_* de módulo, incluindo rh_certificado_colaboradores e rh_listar_planos_acao. Aproveitar para somar AND ativo aos subselects de rh_usuarios. (F2, F3)
P2	Fechar o INSERT em patient_notifications	Trocar WITH CHECK (true) por (false), espelhando doctor_notifications, e revogar INSERT de anon/authenticated. Correção de uma linha. (F6)
Feito	supabase/config.toml versionado com verify_jwt por função	Criado durante esta rodada de correções — a postura de autenticação das 34 funções passou a ser revisável em PR. (F4, F8)
P3	Escapar HTML nos e-mails transacionais	escapeHtml compartilhado em _shared, aplicado a todo valor dinâmico, com allowlist de origem para ctaUrl e rejeição de markup no nome de empresa do autocadastro. (F7)
P3	Autenticar as três funções de cron restantes	Aplicar o guard de expire-credits em send-consultation-reminders, send-glp1-notifications e send-rh-reminders, e condicionar ?forcar=1 ao SERVICE_KEY. (F8)
Feito	user_metadata eliminado de toda policy e função em produção	Os 5 arquivos legados reescritos para chamar is_super_admin(). O smoke test do bloco 4, rodado em produção, achou mais 3 policies vivas com o mesmo problema SEM ARQUIVO correspondente no repositório (consultations x2, plan_prices x1) — essas eram exploráveis de fato, não risco futuro. Corrigidas na mesma migração; hoje nenhuma policy ou função em pg_policies/pg_proc lê user_metadata. (F9)
P3	Controle de migrations aplicadas + smoke test no CI	A causa raiz de F9 segue de pé: migrations aplicadas à mão pelo SQL Editor, sem registro do que já rodou — foi assim que consultations/plan_prices ficaram sem arquivo por tempo indeterminado. Adotar uma tabela de controle de migrations e levar o teste de smoke (já escrito, bloco 4.2) para rodar no CI, não só quando alguém lembra de colar manualmente. (F9)
P3	Automatizar a detecção no CI	Scanner de segredo (gitleaks) no pipeline e um teste que consulte pg_policies procurando USING(true)/WITH CHECK(true) sem cláusula TO restritiva — os dois padrões que produziram F5, F1 e F6.

Issues para o GitHub

Cada bloco abaixo é o texto completo de uma issue em Markdown, pronto para copiar e colar. Achados relacionados foram agrupados numa issue só quando compartilham a mesma correção. Delimitadores: --- ISSUE n --- e --- FIM ISSUE n ---.

--- ISSUE 1 ---

```
## [Segurança] Médico consegue baixar exame de qualquer paciente via UPDATE em
patient_exams.file_url

**Labels:** `security`, `severidade:alta`, `database`, `rls`
**Severidade:** Alta
**Categoria:** 3. IDOR

### Problema

`name` está sem qualificação, e a tabela `doctors` – que está no FROM da própria subquery – TEM
uma coluna `name`. Em SQL, um nome de coluna não qualificado se liga primeiro ao FROM do nível
mais interno. Logo, a policy não compara com `storage.objects.name`: compara com o NOME DO
MÉDICO.

Verificado em PostgreSQL 16, não deduzido. Com o mesmo dado, a versão com `name` nu devolve 0
objetos e a versão com `objects.name` qualificado devolve 1.

Daí saem duas consequências, e a segunda é a grave:

1. Com um nome normal ("Dra Ana Souza"), a condição nunca casa. O médico não abre exame NENHUM –
nem dos próprios pacientes. A visualização de exames do portal está quebrada hoje, e em silêncio:
signStoragePaths engole o erro de createSignedUrls e devolve file_url = null, então a tela mostra
um link inerte em vez de um erro.

2. O EXISTS deixou de ser correlacionado ao objeto. A condição virou "existe algum exame meu cujo
file_url termina com o meu próprio nome" – uma pergunta que não fala do arquivo sendo pedido. Se
ela for verdadeira, é verdadeira para TODOS os objetos do bucket. E o médico controla os dois
lados: basta pôr em `doctors.name` um sufixo de qualquer arquivo dele – "pdf" serve. A policy
doctors_own_update (20260815:31-33) permite editar o próprio nome, e
protect_doctor_privileged_fields (20260815:38-66) protege user_id, status, nível, rating e
platform_fee_percent – mas não o nome.

Confirmado no mesmo teste: com o nome "Dra Ana Souza", 0 de 3 objetos visíveis; após UPDATE do
nome para "pdf", 3 de 3.

Separadamente, patient_exams_doctor_update termina em WITH CHECK (true): o USING filtra a linha
ANTES da escrita, o WITH CHECK valida DEPOIS. O médico reatribui uma linha sua para outro
paciente ou outro profissional. Não é o vetor de leitura, mas passa a ser se alguém corrigir a
policy de storage só qualificando a coluna, sem ancorar na pasta.

### Evidência

`supabase/migrations/20260802_acesso_por_tipo_profissional.sql:175-186`

```sql
CREATE POLICY "doctor_exams_read" ON storage.objects
FOR SELECT TO authenticated
USING (
 bucket_id = 'patient-exams'
AND EXISTS (
 SELECT 1 FROM public.patient_exams pe
 JOIN public.doctors d ON d.id = pe.doctor_id
 WHERE d.user_id = auth.uid()
 AND COALESCE(d.tipo_profissional, 'medico') = 'medico'
)
```

```

 AND pe.file_url LIKE '%' || name -- <= `name` NU
)
);
...

`supabase/migrations/20260422_doctor_panel_v2.sql:357-366`

```sql
CREATE POLICY "patient_exams_doctor_update" ON public.patient_exams
  FOR UPDATE TO authenticated
  USING (
    EXISTS (SELECT 1 FROM public.doctors d
      WHERE d.id = patient_exams.doctor_id AND d.user_id = auth.uid())
  )
  WITH CHECK (true);
...

```

Impacto

Leitura de todos os exames clínicos (PHI) da plataforma por qualquer médico aprovado, ao custo de um UPDATE no próprio perfil. Não é preciso conhecer UUID de vítima nem nome de arquivo: a policy libera o bucket e o storage lista. Violação de LGPD art. 11 e do sigilo profissional. Em paralelo, uma funcionalidade do portal está inoperante desde 20260802.

Condições de explorabilidade

Conta de profissional com status approved, tipo_profissional medico e ao menos um exame vinculado – o fluxo normal do portal já produz isso. Depois, um único UPDATE em doctors.name. Sem feature flag e sem config insegura necessária.

Correção sugerida

1) Qualificar a coluna (storage.objects.name) e trocar o LIKE de sufixo por igualdade. 2) Ancorar o objeto na pasta do paciente da linha: (storage.foldername(storage.objects.name))[1] = pe.patient_id::text – o upload sempre grava em <patient_id>/... e a policy de INSERT obriga a pasta a ser o auth.uid(), então é um vínculo que o médico não move. 3) Preservar a restrição tipo_profissional = 'medico' de 20260802, senão a correção alarga o acesso a psicólogos. 4) WITH CHECK igual ao USING em patient_exams_doctor_update, mais trigger congelando patient_id/doctor_id/file_url. 5) Varrer as demais policies de storage.objects pelo mesmo padrão de sobreposição de coluna.

Critérios de aceite

- [] Policy doctor_exams_read qualifica storage.objects.name e usa igualdade, não LIKE.
- [] Policy doctor_exams_read exige que a pasta do objeto seja o patient_id da linha.
- [] Restrição tipo_profissional = 'medico' preservada (psicólogo não abre exame).
- [] Policy patient_exams_doctor_update tem WITH CHECK equivalente ao USING.
- [] Trigger BEFORE UPDATE impede alteração de patient_id, doctor_id e file_url.
- [] Teste: médico com doctors.name = 'pdf' enxerga só os objetos dos próprios pacientes.
- [] Teste: médico volta a abrir o exame de um paciente seu (a funcionalidade estava quebrada).
- [] Varredura confirma que nenhuma outra policy de storage.objects referencia coluna não qualificada.

Origem: auditoria de segurança de 01 de setembro de 2026 (commit `e94cccf`), achado(s) F1.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Permissões por módulo do portal RH não são verificadas no servidor

```

**Labels:** `security`, `severidade:alta`, `database`, `rls`, `rh`
**Severidade:** Alta
**Categoria:** 2. Permissão definida no navegador

```

Problema

****F2 – Permissões por módulo do portal RH existem só no navegador****

A migration 20260836 introduziu o modelo de permissões por módulo (colaboradores, saude_mental, absenteismo, plano_acao, importar, financeiro, compliance, empresa, usuarios, apuracao) e o helper `rh_tem_permissao()`. Esse helper foi de fato aplicado nas policies de empresa_colaboradores (20260836:196-221), nas RPCs de relatos confidenciais, nas de liderança e nas duas financeiras de 20260843.

Fora dessas, o servidor continua perguntando apenas 'este usuário é um `rh_usuarios` desta empresa?'. Ficaram sem o gate de módulo:

- `empresa_afastamentos` (SELECT/INSERT/DELETE) – CID e dias de afastamento;
- `empresa_ambulatorio`, `empresa_planos_acao`, `empresa_compliance_docs`, `empresa_certificados_esg`, `empresa_ingestao_lotes`, `empresa_relatorios_emitidos`, `psychosocial_campaigns`, `empresa_faturas` – todas com policy só de empresa;
- RPCs `rh_relatorio_psicossocial`, `rh_matriz_psicossocial`, `rh_relatorio_jss`, `rh_absenteismo_resumo`, `rh_ambulatorio_resumo`, `rh_listar_planos_acao`, `rh_criar/atualizar/excluir_plano_acao`, `rh_lancar_afastamento`, `rh_lancar_ambulatorio`, `rh_vincular_cpfs`, `rh_ingerir_afastamentos`.

O bloqueio é só a UI: `RhLayout` filtra as abas e `RhPermissionGate` redireciona a rota. Quem tem sessão do portal já tem a anon key no bundle e pode chamar `supabase.from('empresa_afastamentos').select('*')` ou `supabase.rpc('rh_relatorio_psicossocial', ...)` direto do console do navegador.

****F3 – `rh_certificado_colaboradores()` contorna a policy do módulo 'colaboradores'**

Esta é a única tabela onde a permissão de módulo FOI aplicada em RLS (20260836:196-201 exige `rh_tem_permissao('colaboradores')`). Mas esta RPC é SECURITY DEFINER e ignora RLS por definição, e o único guard que ela faz é 'existe uma linha em `rh_usuarios` com meu `user_id`'. Ela nem sequer checa 'ativo'. O resultado é a lista nominal completa de colaboradores da empresa – nome, setor, função, datas de entrada e saída – para quem a policy da tabela acabou de negar.

O mesmo padrão aparece em `rh_listar_planos_acao` (20260847_plano_expoe_origem_lideranca.sql:51), que também é SECURITY DEFINER sem checagem de módulo nem de conta ativa.

Evidência

```
`src/contexts/RhAccessContext.tsx:15,26-28`
```

```
````ts
can: permissao => acesso.principal || acesso.permissoes.includes(permissao),
...
export const RhPermissionGate: React.FC<{...}> = ({ permissao, children }) => {
 const { can } = useRhAccess();
 return can(permissao) ? <>{children}</> : <RhHomeRedirect />;
};
````
```

```
`supabase/migrations/20260731_absenteismo_ambulatorio.sql:66-69`
```

```
````sql
CREATE POLICY "rh reads own afastamentos"
 ON empresa_afastamentos FOR SELECT TO authenticated
 USING (empresa_id IN (SELECT empresa_id FROM rh_usuarios WHERE user_id = auth.uid()));
````
```

```
`supabase/migrations/20260803_link_por_setor.sql:512-517`
```

```
````sql
SELECT e.id, e.nome, e.cnpj INTO v_empresa
FROM empresas e
WHERE e.id IN (SELECT empresa_id FROM rh_usuarios WHERE user_id = auth.uid())
LIMIT 1;
```

```
IF v_empresa.id IS NULL THEN RETURN NULL; END IF;
```


`supabase/migrations/20260842_relatorios_valor_probatorio.sql:242-275`


```

```
```sql
CREATE OR REPLACE FUNCTION rh_certificado_colaboradores()
RETURNS TABLE (
 colaborador_id UUID, nome TEXT, setor TEXT, funcao TEXT,
 data_adicao TIMESTAMPTZ, data_ativacao TIMESTAMPTZ,
 data_saida TIMESTAMPTZ, status TEXT
) AS $$
WITH emp AS (
 SELECT empresa_id FROM rh_usuarios WHERE user_id = auth.uid() LIMIT 1
),
...
FROM empresa_colaboradores ec
LEFT JOIN public.profiles p ON p.id = ec.user_id
LEFT JOIN auth.users u ON u.id = ec.user_id
WHERE ec.empresa_id = (SELECT empresa_id FROM emp)
$$ LANGUAGE sql SECURITY DEFINER SET search_path = public STABLE;
```
```

Impacto

Escalada horizontal dentro da empresa. Um membro com papel 'financeiro' lê o relatório psicossocial (WHO-5/JSS por setor), a lista de afastamentos com CID e os atendimentos de ambulatório – exatamente o dado que o desenho do produto separa por módulo. O modelo de permissão da tela de Usuários passa a ser uma promessa que o servidor não cumpre.

Anula, na prática, a única policy de módulo implementada do lado servidor. Exposição da nominata de colaboradores (dado pessoal identificável) a membros do RH que a empresa decidiu não dar esse acesso.

Condições de explorabilidade

Qualquer conta de equipe do RH criada via invite-rh-user com permissões restritas. Nenhuma condição especial: basta a sessão legítima e uma chamada direta ao PostgREST/RPC. O corte k-anônimo ($k \geq 5$) segue valendo e limita a reidentificação individual, mas não impede a leitura do módulo inteiro.

Uma chamada supabase.rpc('rh_certificado_colaboradores') com a sessão de qualquer rh_usuarios da empresa, inclusive uma conta desativada cujo user_id não tenha sido zerado.

Correção sugerida

Adicionar `public.rh\_tem\_permissao('<módulo>')` a cada policy e a cada RPC listada acima, no mesmo formato já usado em empresa_colaboradores. Aproveitar para incluir `AND ativo` no subselect de rh_usuarios – várias policies e RPCs não filtram por conta ativa (a desativação hoje depende de rh_atualizar_usuario zerar o user_id, que é uma proteção indireta).

Adicionar no topo da função o mesmo guard das RPCs de relato: selecionar de rh_usuarios com `AND ativo AND (principal OR 'colaboradores' = ANY(permissoes))` e levantar exceção quando não houver linha. Repetir em rh_listar_planos_acao com o módulo 'plano_acao'.

Critérios de aceite

- [] Toda policy de tabela empresa_* chama rh_tem_permissao() com o módulo correspondente.
- [] Toda RPC rh_* de leitura/escrita de módulo valida a permissão antes de consultar.
- [] Subselects de rh_usuarios em policies incluem AND ativo.
- [] Teste: usuário com permissoes=['financeiro'] recebe 0 linhas em empresa_afastamentos e exceção em rh_relatorio_psicossocial.
- [] Teste: usuário principal continua vendo todos os módulos.
- [] rh_certificado_colaboradores() levanta exceção para usuário sem 'colaboradores'.
- [] rh_certificado_colaboradores() levanta exceção para usuário com ativo = false.
- [] rh_listar_planos_acao() aplica o mesmo guard com o módulo 'plano_acao'.

- [] Emissão do certificado de disponibilização continua funcionando para o usuário principal.

Origem: auditoria de segurança de 01 de setembro de 2026 (commit `e94cccf`), achado(s) F2, F3.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança][Resolvido] strava-webhook – descomissionado em vez de autenticado

Labels: `security`, `severidade:alta`, `edge-functions`, `resolvido`

Severidade: Alta

Categoria: 3. IDOR

Problema

O ramo GET valida STRAVA_VERIFY_TOKEN (linha 16) – é o handshake de assinatura. O ramo POST, que é o que efetivamente escreve no banco, não valida absolutamente nada: nem token, nem assinatura, nem origem. O cliente Supabase é instanciado com SUPABASE_SERVICE_ROLE_KEY (linha 41), então processEvent roda com poder total, acima de qualquer RLS.

Todo o roteamento usa campos do corpo enviado pelo atacante: `owner\_id` decide de qual usuário a conexão será apagada, e `object\_id` decide qual atividade será buscada e gravada. É o padrão clássico de IDOR – identificador de objeto vindo do request, sem verificação de quem está chamando.

Evidência

`supabase/functions/strava-webhook/index.ts:24-35`

```
``ts
// -- POST: Evento de atividade ou deautorização -----
if (req.method === 'POST') {
  // Strava exige resposta em 2 segundos – processamento em background
  const payload = await req.json();

  // Responder imediatamente antes de processar
  const responsePromise = new Response('EVENT_RECEIVED', { status: 200 });

  // Processar em background (não bloqueia a resposta)
  processEvent(payload).catch(...);

  return responsePromise;
}
...`
```

`supabase/functions/strava-webhook/index.ts:46-57`

```
``ts
const ownerId    = payload.owner_id as number;

// -- Deautorização: remover conexão -----
if (objectType === 'athlete') {
  const updates = payload.updates as Record<string, string> | undefined;
  if (updates?.authorized === 'false') {
    await supabase
      .from('strava_connections')
      .delete()
      .eq('strava_athlete_id', ownerId);
  }
  return;
}
...`
```

Impacto

Negação de serviço direcionada: POST com {object_type:'athlete', owner_id:N, updates:{authorized:'false'}} apaga a conexão Strava do usuário N; iterando N apaga todas. Além disso, o mesmo endpoint permite forçar a re-sincronização e a gravação de atividades em

activities e flow_stats de terceiros – dados que alimentam os relatórios clínicos de IA (generate-patient-ai-report e activity-health-insights) e, portanto, contaminam conduta médica.

Condições de explorabilidade

Depende da flag verify_jwt com que a função foi publicada – e o repositório NÃO declara essa flag (não existe supabase/config.toml). Nos dois cenários há exposição: com --no-verify-jwt (obrigatório para o Strava alcançar a função, e é o que a própria webhook-asaas documenta na linha 310) qualquer pessoa na internet explora; com verify_jwt padrão, qualquer usuário logado do app explora. O apagamento de conexões (object_type 'athlete') não requer nem token do Strava.

Correção sugerida

RESOLVIDO POR DESCOMISSIONAMENTO, não por autenticação (decisão de produto de 01/09/2026): a plataforma passou a cobrir atividade física por Apple HealthKit e Google Health Connect, então proteger um endpoint que ninguém mais deveria chamar deixou de fazer sentido – desligar é mais seguro que autenticar algo obsoleto. strava-webhook agora devolve 410 Gone incondicionalmente, sem ler o corpo da requisição nem tocar em strava_connections/activities. strava-oauth e strava-oauth-callback (que abriam novas conexões) foram desativadas junto, pelo mesmo motivo. strava-sync e strava-refresh-token permanecem ativas apenas para quem já tinha o Strava conectado, até a remoção completa da integração (tarefa separada: 18 arquivos de frontend, as 5 Edge Functions, migrations e a decisão sobre o destino de strava_connections e das activities já sincronizadas).

Critérios de aceite

- [] strava-webhook devolve 410 para GET e POST, sem exceção.
- [] strava-oauth e strava-oauth-callback devolvem 410, sem executar troca de OAuth.
- [] Nenhuma das três funções acima lê strava_connections, activities ou flow_stats.
- [] strava-sync e strava-refresh-token continuam funcionando para conexões existentes.
- [] Teste: POST arbitrário em strava-webhook não altera nenhuma tabela.

Origem: auditoria de segurança de 01 de setembro de 2026 (commit `e94cccf`), achado(s) F4.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança][Resolvido] STRAVA_VERIFY_TOKEN – urgência de rotação superada pelo descomissionamento

****Labels:**** `security`, `severidade:media`, `secrets`, `resolvido`
****Severidade:**** Média
****Categoria:**** 4. Chaves expostas

Problema

STRAVA_CLIENT_SECRET foi corretamente deixado em branco, com aviso explícito. STRAVA_VERIFY_TOKEN, não: o valor real de produção aparece três vezes no arquivo, inclusive como valor efetivo na linha 50 e dentro do comando de provisionamento da linha 43. O arquivo é rastreado pelo git desde 37f6380 (14/03/2026) – confirmado com `git log --diff-filter=A -- '\*.env\*`.

Esse token é o único fator que autentica o handshake de assinatura do webhook (strava-webhook/index.ts:16). Ele não é um placeholder: segue o padrão de nome do projeto ('nura' era o nome anterior do produto) e casa com o comando de provisionamento documentado.

Evidência

`env.example:43-50`

```
``ts
# Configure via: supabase secrets set STRAVA_CLIENT_ID=227202 STRAVA_CLIENT_SECRET=xxx
STRAVA_VERIFY_TOKEN=nura_strava_2026
#
# STRAVA_CLIENT_ID      = 227202
# STRAVA_CLIENT_SECRET  = <seu client secret do portal Strava – não committar>
# STRAVA_VERIFY_TOKEN   = nura_strava_2026
```

```
STRAVA_CLIENT_ID=227202
STRAVA_CLIENT_SECRET=
STRAVA_VERIFY_TOKEN=nura_strava_2026
```

```

### ### Impacto

Quem tem acesso ao repositório (ou a qualquer fork/clone/backup dele) pode registrar sua própria subscription no endpoint e responder ao handshake de validação. Combinado com F4, remove a última barreira de obscuridade do webhook. Vale também como sinal de processo: não existe validação de startup que recuse um segredo igual ao default publicado.

### ### Condições de explorabilidade

Imediata para qualquer pessoa com leitura do repositório. Não exige nenhuma condição de configuração.

### ### Correção sugerida

Ficou parcialmente superado pela decisão de descomissionar o webhook (ver F4): o segredo que vazou não protege mais nada, porque strava-webhook não lê STRAVA\_VERIFY\_TOKEN em nenhuma circunstância – a função devolve 410 antes de examinar qualquer header ou query string. O valor não precisa mais ser rotacionado por urgência de segurança. .env.example foi limpo (não cita mais STRAVA\_VERIFY\_TOKEN nem STRAVA\_WEBHOOK\_SECRET). Continua valendo, como prevenção geral: guard de inicialização nas Edge Functions que recusa segredo igual a um valor conhecido de exemplo, e scanner de segredo no CI (P3).

### ### Critérios de aceite

- [ ] strava-webhook não lê STRAVA\_VERIFY\_TOKEN em nenhum caminho de código (verificado).
- [ ] .env.example não contém nenhum valor de segredo real (só placeholders).
- [ ] Rotacionar STRAVA\_VERIFY\_TOKEN deixou de ser urgência; opcional, útil só se a integração for reativada antes da remoção completa.
- [ ] Existe checagem de startup que falha alto se um segredo estiver ausente ou for o default de exemplo.
- [ ] Scanner de segredo (gitleaks ou equivalente) rodando no CI.

\_Origem: auditoria de segurança de 01 de setembro de 2026 (commit `e94cccf`), achado(s) F5.\_

--- FIM ISSUE 4 ---

## --- ISSUE 5 ---

## [Segurança] Qualquer usuário autenticado insere notificações na caixa de outro paciente

```
Labels: `security`, `severidade:media`, `database`, `rls`
Severidade: Média
Categoria: 1. Banco sem tranca (isolamento)
```

### ### Problema

A intenção declarada no comentário é 'service role pode inserir'. Mas a policy não tem cláusula TO, então vale para todos os papéis, incluindo authenticated e anon – e o Supabase concede INSERT nessas tabelas por padrão (não há REVOKE para patient\_notifications em nenhuma migration). O WITH CHECK (true) não amarra o user\_id ao auth.uid().

Vale notar o contraste: a tabela irmã doctor\_notifications faz certo – 20260422\_doctor\_panel\_notifications.sql:60 usa WITH CHECK (false), permitindo insert apenas via SECURITY DEFINER. O comentário nessa linha é literalmente 'apenas service\_role insere'. A regra correta já existe no projeto; só não foi aplicada aqui.

Além disso, a policy era desnecessária desde o início: os triggers que a justificam (notify\_patient\_on\_chat\_opened, linha 50) são SECURITY DEFINER e escrevem como o dono da função, não como o usuário.

### ### Evidência

```
`supabase/migrations/20260423_patient_notifications.sql:42-46`
```

```
```sql
-- Service role pode inserir (usado pelos triggers SECURITY DEFINER)
CREATE POLICY "patient_notifications_insert_service"
  ON public.patient_notifications
  FOR INSERT
  WITH CHECK (true);
```
```

### ### Impacto

Phishing dentro do produto. O atacante escolhe title, body e o JSONB `data` de uma notificação exibida ao paciente com a credibilidade da própria plataforma – por exemplo, tipo 'chat\_opened' apontando para um chat que não existe, ou um aviso clínico falso. Também permite poluir e inundar a caixa de notificações de qualquer usuário.

### ### Condições de explorabilidade

Uma chamada `supabase.from('patient_notifications').insert({...})` com qualquer sessão autenticada do app. Sem pré-condição. A leitura permanece corretamente restrita ao dono (policy de SELECT na linha 31), então não há vazamento – o problema é de escrita.

### ### Correção sugerida

Trocar por `WITH CHECK (false)`, espelhando `doctor_notifications`, ou por `TO service\_role WITH CHECK (true)`. Confirmar que os triggers SECURITY DEFINER continuam inserindo (eles não são afetados por RLS quando o dono é superusuário/postgres) e adicionar `REVOKE INSERT ON public.patient_notifications FROM anon, authenticated`.

### ### Critérios de aceite

- [ ] Policy de INSERT em `patient_notifications` não permite escrita por `authenticated/anon`.
- [ ] `REVOKE INSERT` explícito para `anon` e `authenticated` aplicado.
- [ ] Triggers `notify_patient_on_chat_opened` e correlatos continuam gerando notificação.
- [ ] Teste: `insert` direto via PostgREST com sessão de paciente retorna 42501.

\_Origem: auditoria de segurança de 01 de setembro de 2026 (commit `e94cccf`), achado(s) F6.\_

--- FIM ISSUE 5 ---

## --- ISSUE 6 ---

## [Segurança] E-mails transacionais interpolam dado de origem pública em HTML sem escape

```
Labels: `security`, `severidade:media`, `edge-functions`
Severidade: Média
Categoria: 5. Inputs sem tratamento (XSS)
```

### ### Problema

`brandedEmailHtml` interpola `heading`, `bodyParagraphs`, `eyebrow`, `footnote`, `ctaText` e `ctaUrl` direto no template, sem escape. Não existe nenhuma função de sanitização no módulo – o projeto tem uma (`escapeHtml` em `api/share.ts:28-35`), mas ela vive do outro lado e não é reutilizada aqui.

O valor que chega até esse HTML nem sempre é confiável. `self-register-empresa` é uma função PÚBLICA e não autenticada: ela grava `empresas.nome` com o que veio do formulário, sem qualquer filtro de markup (a única validação é 'não vazio'). Esse mesmo campo depois vai para `activationEmailHtml` em `invite-colaborador:196` e `resend-invite:79`, e para o alerta ao administrador em `webhook-asaas:295`.

É importante ser preciso sobre o alcance: cliente de e-mail moderno não executa JavaScript, então isto não é XSS clássico. O que se obtém é injeção de HTML – âncoras, imagens de rastreo e blocos de texto forjados dentro de uma mensagem com a marca Malama.

### ### Evidência

```
`supabase/functions/_shared/emails.ts:42-47`

````ts
const paragraphs = bodyParagraphs
  .map(
    (p) =>
      `
```

--- ISSUE 7 ---

[Segurança] Funções de cron não verificam o chamador e verify_jwt não é versionado

Labels: `security`, `severidade:baixa`, `edge-functions`, `infra`

Severidade: Baixa

Categoria: Extra: autorização de superfície

Problema

Três das funções acionadas por pg_cron descartam a requisição inteira (o parâmetro se chama `\_req`) ou leem só a query string, sem conferir o Authorization. Isso destoa do padrão correto adotado no resto do projeto: expire-credits:19-22, compute-clinical-outcomes:33-38, generate-empirical-cases:422-439 e process-payouts:360-368 todas comparam o token com o SERVICE_KEY ou exigem super_admin.

send-rh-reminders é a mais sensível das três porque expõe `?forcar=1`, que ignora a checagem de dia da semana.

Evidência

`supabase/functions/send-consultation-reminders/index.ts:247`

```ts

```
Deno.serve(async (_req) => {
 try {
 const nowMs = Date.now();
 ...
```

`supabase/functions/send-rh-reminders/index.ts:210-212`

```ts

```
Deno.serve(async (req) => {
  // `?forcar=1` ignora o dia da semana – para testar sem esperar a segunda.
  const url = new URL(req.url);
  const forcar = url.searchParams.get('forcar') === '1';
  ...
```

Impacto

Disparo forçado de rotinas de envio: lembretes de consulta, notificações de dose GLP-1 e o digest semanal do RH. O dedupe interno (consultation_reminders_sent, notification_sent, rh_lembretes_enviados) limita o volume a um envio por evento, então o efeito é antecipação e ruído, não flood. send-consultation-reminders também executa processNoShows, que marca consultas como no_show e devolve crédito – rodá-la fora de hora antecipa essa transição de estado.

Condições de explorabilidade

Depende de verify_jwt, que o repositório não versiona (não há supabase/config.toml). Com o padrão verify_jwt=true, qualquer usuário logado consegue invocar. Com --no-verify-jwt, qualquer pessoa. O impacto contido pelo dedupe é o que mantém isto em severidade baixa.

Correção sugerida

Aplicar o guard de expire-credits/index.ts:19-22 nas três funções e versionar supabase/config.toml declarando verify_jwt por função, para que a postura pare de depender de uma flag de linha de comando não registrada.

Critérios de aceite

- [] As três funções recusam chamada sem o SERVICE_KEY (ou sem papel super_admin).
- [] supabase/config.toml versionado, com verify_jwt explícito para cada função.
- [] O parâmetro ?forcar=1 só é aceito junto do SERVICE_KEY.
- [] Os jobs pg_cron continuam executando normalmente após a mudança.

Origem: auditoria de segurança de 01 de setembro de 2026 (commit `e94cccf`), achado(s) F8.

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

[Segurança] Migrations legadas ainda checam papel por user_metadata (risco de reintrodução)

Labels: `security`, `severidade:baixa`, `database`, `tech-debt`

Severidade: Baixa

Categoria: 2. Permissão definida no navegador

Problema

user_metadata é gravável pelo próprio usuário (supabase.auth.updateUser). Qualquer verificação de privilégio que o leia é auto-promoção. A migration 20260626_rbac_app_metadata.sql fechou isso corretamente: redefiniu is_super_admin() para ler app_metadata (linha 23-28) e refez, com o mesmo nome e a mesma assinatura, todas as policies e RPCs afetadas – incluindo 'Admins manage payouts' (linha 96-99), 'Admins can manage settings' (90-93), 'Admins can view all profiles' (31-34), admin_get_all_users() (37-66) e admin_update_setting() (70-87).

O achado não é uma brecha aberta hoje: é o risco de reintrodução. Os CINCO arquivos vulneráveis continuam no diretório de migrations, e este projeto aplica migrations manualmente pelo SQL Editor, arquivo a arquivo, sem tabela de controle de versão aplicada. Executar qualquer um deles depois de 20260626 – num ambiente novo, num restore, ou por engano de ordenação – recria a policy vulnerável em silêncio, sem erro.

CONFIRMADO NA PRÁTICA em 01/09/2026: ao aplicar a correção deste relatório, o teste de smoke do bloco 4 (que varre pg_policies e pg_proc inteiros, não só os cinco arquivos catalogados) achou TRÊS policies vivas com o mesmo predicado – 'Admin can view all consultations', 'Admin can update all consultations' e 'plan_prices_admin_write' – que não existem em NENHUM arquivo do repositório, nem em supabase/migrations/, nem nos supabase-*.sql anteriores à pasta de migrations. Foram aplicadas diretamente no banco em algum momento não documentado. Ou seja: a classificação original – 'risco de reintrodução, não brecha ativa' – estava certa para os cinco arquivos catalogados, mas incompleta: para consultations e plan_prices, a auto-promoção via user_metadata era EXPLORÁVEL AGORA, não uma possibilidade futura. As três foram corrigidas na mesma migração (bloco 4) e o smoke test hoje passa limpo.

Evidência

`supabase/migrations/20260621\_payouts\_doctor\_policy.sql:27-30`

```
```sql
CREATE POLICY "Admins manage payouts"
 ON payouts FOR ALL
 USING ((auth.jwt() -> 'user_metadata' ->> 'role') = 'super_admin')
 WITH CHECK ((auth.jwt() -> 'user_metadata' ->> 'role') = 'super_admin');
```
```

`supabase/migrations/20260601\_empresas\_b2b.sql:21-24`

```
```sql
-- Helper: o JWT do chamador é de um super_admin?
CREATE OR REPLACE FUNCTION is_super_admin()
RETURNS BOOLEAN AS $$
 SELECT (auth.jwt() -> 'user_metadata' ->> 'role') = 'super_admin';
$$ LANGUAGE sql STABLE;
```
```

`supabase/migrations/20260626\_rbac\_app\_metadata.sql:5-8 (a correção)`

```
```sql
-- PROBLEMA: o papel (super_admin/rh) era lido de user_metadata, que o próprio
-- usuário altera via supabase.auth.updateUser({ data: { role } }) no navegador.
-- Isso permitia auto-promoção a super_admin (acesso a todos os dados).
-- app_metadata (raw_app_meta_data) só é gravável por service_role / admin API.
```
```

`pg\_policies em produção – sem arquivo correspondente em nenhum lugar do repo`

```
```ts
CREATE POLICY "Admin can view all consultations" ON consultations FOR SELECT
```

```

 USING (((auth.jwt() -> 'user_metadata') ->> 'role') = 'super_admin');
CREATE POLICY "Admin can update all consultations" ON consultations FOR UPDATE
 USING (((auth.jwt() -> 'user_metadata') ->> 'role') = 'super_admin');
CREATE POLICY plan_prices_admin_write ON plan_prices FOR ALL
 USING (((auth.jwt() -> 'user_metadata') ->> 'role') = 'super_admin');
...

```

### ### Impacto

O pior dos cinco é 20260601\_empresas\_b2b.sql:21-24: ali não é uma policy, é a DEFINIÇÃO de `is_super_admin()` – o ponto único que todas as policies administrativas consultam. Reaplicar aquele arquivo reabre empresas, payouts, platform\_settings e profiles de uma vez só, sem erro nenhum.

Se reintroduzido: escalada a `super_admin` por qualquer usuário do app, com controle sobre `platform_settings` (valores de repasse por nível, taxa de transação) e sobre a tabela `payouts`. Para os cinco arquivos catalogados, o risco era de processo, não de estado corrente – mas ver a nota acima: para `consultations` (dados de consulta/telemedicina) e `plan_prices` (preços dos planos), o mesmo problema era estado corrente, não hipótese, até ser corrigido nesta rodada.

### ### Condições de explorabilidade

Para os cinco arquivos catalogados: não explorável no estado em que a auditoria original encontrou o banco, desde que 20260626 tenha sido a última a rodar sobre esses objetos – torna-se explorável em qualquer reaplicação fora de ordem. Para as três policies descobertas depois (`consultations x2`, `plan_prices x1`): eram exploráveis sem nenhuma condição – bastava `supabase.auth.updateUser({ data: { role: 'super_admin' } })` no navegador. Corrigido nesta migração.

### ### Correção sugerida

Reescrever os quatro arquivos legados para chamar `is_super_admin()` (tornando-os idempotentes e seguros em qualquer ordem) ou marcá-los como superados com um cabeçalho e um ``RAISE EXCEPTION`` de guarda. Adotar uma tabela de migrations aplicadas para que a ordem deixe de ser responsabilidade humana. Adicionar um teste de smoke que verifique que nenhuma policy em `pg_policies` referencia `user_metadata`.

### ### Critérios de aceite

- [ ] Nenhum arquivo em `supabase/migrations/` contém checagem de papel via `user_metadata`.
- [ ] Query sobre `pg_policies` não retorna nenhuma policy que referencia `user_metadata` – confirmado (consulta rodada em produção, 01/09/2026).
- [ ] `consultations` e `plan_prices` usam `is_super_admin()`, como o resto do projeto – confirmado.
- [ ] Existe controle de migrations aplicadas (tabela ou ferramenta), não só ordem de nome de arquivo.
- [ ] Teste de smoke no CI verifica a ausência de `user_metadata` em policies e funções.
- [ ] Auditoria de schema completo (`pg_dump --schema-only` ou equivalente) para achar outros objetos vivos sem arquivo correspondente no repositório – este achado mostrou que existem.

\_Origem: auditoria de segurança de 01 de setembro de 2026 (commit `e94cccf`), achado(s) F9.\_

--- FIM ISSUE 8 ---